

Proyecto 1: Clasificación de Contaminaciones
Detección de Granos de Arroz mediante
Aprendizaje Automático Clásico

IE0435 - Inteligencia Artificial

Nombre: Luis Adrián Aguirre

Carné: C10153

Profesor: Marvin Coto Jiménez

Escuela de Ingeniería Eléctrica

Universidad de Costa Rica

I Ciclo 2026

Índice

1. Introducción	4
1.1. Contexto	4
1.2. Objetivo	4
1.3. Alcance del Reporte	4
2. Metodología	4
2.1. Conjunto de Datos	4
2.2. Preprocesamiento	5
2.3. División de Datos	5
2.4. Modelos Evaluados	5
2.5. Optimización de Hiperparámetros	5
2.5.1. Árbol de Decisión	6
2.5.2. Naive Bayes	6
2.5.3. K-Nearest Neighbors	6
2.5.4. Support Vector Machine	6
2.6. Métricas de Evaluación	6
3. Resultados	7
3.1. Comparación de Modelos	7
3.2. Análisis por Modelo	7
3.2.1. Árbol de Decisión (Modelo Seleccionado)	7
3.2.2. Naive Bayes	8
3.2.3. K-Nearest Neighbors	8
3.2.4. Support Vector Machine	9
3.3. Justificación de la Selección del Modelo	9
3.4. Análisis de Errores	10
3.5. Limitaciones del Estudio	10
4. Exportación del Modelo	10
4.1. Formato de Exportación	10
4.2. Información del Archivo	11
4.3. Uso del Modelo Exportado	11
4.4. Requisitos del Sistema	11
5. Conclusiones	12
5.1. Logros del Proyecto	12
5.2. Recomendaciones para Trabajo Futuro	12
5.3. Lecciones Aprendidas	13
6. Referencias	13
A. Especificaciones del Hardware Utilizado	14
B. Tiempos de Ejecución	14

C. Metadatos del Modelo Exportado

14

1. Introducción

1.1. Contexto

En procesos de manufactura modernos, la inspección visual automática juega un papel crucial en la detección y cuantificación de defectos o anomalías para asegurar la calidad del producto final. Este proyecto simula un sistema de inspección visual para detectar “contaminaciones” (granos de arroz) en una línea de producción mediante técnicas de aprendizaje automático clásico aplicadas al procesamiento de imágenes.

1.2. Objetivo

El objetivo principal es construir un sistema completo de clasificación binaria que incluya:

- Recolección y preprocesamiento de datos (imágenes)
- Construcción de un conjunto de datos etiquetado
- Entrenamiento y evaluación de múltiples modelos de clasificación
- Selección del mejor modelo mediante optimización de hiperparámetros
- Exportación del modelo final para su implementación

1.3. Alcance del Reporte

Este documento se enfoca específicamente en las etapas de **modelado** (Punto 3.4) y **exportación** (Punto 3.5) del proyecto, presentando:

1. Metodología de entrenamiento
2. Comparación de modelos de clasificación clásica
3. Optimización de hiperparámetros
4. Evaluación de métricas de desempeño
5. Selección y exportación del modelo final

2. Metodología

2.1. Conjunto de Datos

El conjunto de datos utilizado fue generado mediante el procesamiento descrito en los puntos 3.2 y 3.3 del proyecto. Después de la limpieza y eliminación de duplicados, el dataset final presentó las siguientes características:

- **Características originales:** 16,384 features (píxeles de imágenes 128×128 binarizadas)

- **Reducción de dimensionalidad:** Se aplicó PCA (Análisis de Componentes Principales) reduciendo a 5 componentes, explicando el 53.88 % de la varianza total
- **Clases:**
 - Clase 1 (positiva): Imágenes con granos de arroz (12 muestras, 44.4 %)
 - Clase 0 (negativa): Imágenes sin arroz (15 muestras, 55.6 %)
- **Total de muestras únicas:** 27 (de 60 originales, se eliminaron 33 duplicados)

2.2. Preprocesamiento

El preprocesamiento incluyó las siguientes etapas:

1. **Limpieza de duplicados:** Eliminación de muestras idénticas
2. **Reducción de dimensionalidad:** PCA con 5 componentes
3. **Normalización:** Los datos ya estaban binarizados (0 y 1)
4. **División estratificada:** 80 % entrenamiento, 20 % prueba

2.3. División de Datos

Se aplicó una división estratificada para mantener la proporción de clases:

- **Conjunto de entrenamiento:** 21 muestras (80 %)
- **Conjunto de prueba:** 6 muestras (20 %)
- **Random state:** 42 (para reproducibilidad)

2.4. Modelos Evaluados

Se entrenaron cuatro algoritmos de clasificación clásica:

1. **Árbol de Decisión** (Decision Tree)
2. **Naive Bayes Gaussiano**
3. **K-Nearest Neighbors** (KNN)
4. **Support Vector Machine** (SVM)

2.5. Optimización de Hiperparámetros

Para cada modelo se realizó una búsqueda en grilla (*Grid Search*) con validación cruzada de 5 pliegues (5-fold cross-validation) para encontrar la mejor combinación de hiperparámetros.

2.5.1. Árbol de Decisión

Espacio de búsqueda:

```

1 param_grid = {
2     'max_depth': [3, 5, 7, 10],
3     'min_samples_split': [2, 3, 5],
4     'criterion': ['gini', 'entropy']
5 }
```

Listing 1: Hiperparámetros - Árbol de Decisión

2.5.2. Naive Bayes

Naive Bayes se entrenó con los parámetros por defecto de scikit-learn.

2.5.3. K-Nearest Neighbors

Espacio de búsqueda:

```

1 param_grid = {
2     'n_neighbors': [3, 5, 7],
3     'weights': ['uniform', 'distance']
4 }
```

Listing 2: Hiperparámetros - KNN

2.5.4. Support Vector Machine

Espacio de búsqueda:

```

1 param_grid = {
2     'C': [0.1, 1, 10],
3     'kernel': ['linear', 'rbf']
4 }
```

Listing 3: Hiperparámetros - SVM

2.6. Métricas de Evaluación

Se utilizaron las siguientes métricas para evaluar el desempeño:

- **Accuracy** (Exactitud): Proporción de predicciones correctas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Precision** (Precisión): Proporción de predicciones positivas correctas

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

- **Recall** (Sensibilidad): Proporción de positivos detectados

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

- **F1-Score**: Media armónica de Precision y Recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

- **Matriz de Confusión**: Para análisis detallado de errores

donde:

- TP = Verdaderos Positivos
- TN = Verdaderos Negativos
- FP = Falsos Positivos
- FN = Falsos Negativos

3. Resultados

3.1. Comparación de Modelos

La Tabla 1 presenta un resumen comparativo del desempeño de los cuatro modelos entrenados después de aplicar PCA.

Cuadro 1: Comparación de métricas de los modelos entrenados

Modelo	Accuracy	Precision	Recall	F1-Score
Árbol de Decisión	0.8333	0.8571	0.8571	0.8571
Naive Bayes	0.8333	0.8571	0.8571	0.8571
KNN	0.6667	1.0000	0.5714	0.7500
SVM	0.8333	0.8571	0.8571	0.8571

3.2. Análisis por Modelo

3.2.1. Árbol de Decisión (Modelo Seleccionado)

Mejores hiperparámetros encontrados:

- max_depth: 3
- min_samples_split: 5
- criterion: gini

Matriz de confusión:

		Predicho	
		Negativo	Positivo
Real	Negativo	4	1
	Positivo	0	1

Cuadro 2: Matriz de confusión - Árbol de Decisión (sobre 6 muestras de prueba)

Métricas detalladas:

- Verdaderos Negativos (TN): 4
- Falsos Positivos (FP): 1
- Falsos Negativos (FN): 0
- Verdaderos Positivos (TP): 1

3.2.2. Naive Bayes**Resultados:**

- Accuracy: 0.8333 (igual al Árbol de Decisión)
- F1-Score: 0.8571
- Tiempo de entrenamiento: Muy rápido (sin búsqueda de hiperparámetros extensiva)

Observaciones:

- Naive Bayes asume independencia entre features, lo cual no es estrictamente cierto para píxeles de imágenes
- A pesar de esto, logró un desempeño competitivo gracias a la reducción con PCA

3.2.3. K-Nearest Neighbors**Mejores hiperparámetros encontrados:**

- `n_neighbors`: 3
- `weights`: uniform

Resultados:

- Accuracy: 0.6667 (el más bajo de los cuatro modelos)
- Precision: 1.000 (no tuvo falsos positivos)
- Recall: 0.5714 (baja capacidad de detectar positivos)
- F1-Score: 0.7500

Observaciones:

- KNN fue el modelo con peor desempeño
- Sensible al tamaño reducido del dataset (solo 21 muestras de entrenamiento)
- La distancia euclidiana en espacios de baja dimensionalidad (5 componentes PCA) puede no ser óptima

3.2.4. Support Vector Machine**Mejores hiperparámetros encontrados:**

- C: 10
- kernel: rbf

Resultados:

- Accuracy: 0.8333
- F1-Score: 0.8571
- Mismo desempeño que Árbol de Decisión y Naive Bayes

3.3. Justificación de la Selección del Modelo

El modelo **Árbol de Decisión** fue seleccionado como el mejor modelo por las siguientes razones:

1. **Alto desempeño:** Accuracy de 83.33 % y F1-Score de 0.8571, igualando a Naive Bayes y SVM
2. **Interpretabilidad:** Los árboles de decisión son altamente interpretables, permitiendo entender exactamente cómo se toman las decisiones de clasificación
3. **Simplicidad:** Con solo profundidad 3 (`max_depth=3`), el árbol es simple y evita sobreajuste
4. **Eficiencia computacional:** Tiempo de entrenamiento y predicción muy rápidos
5. **Robustez ante el tamaño del dataset:** A diferencia de KNN, el árbol de decisión no se vio tan afectado por el número reducido de muestras
6. **Balance perfecto:** No presentó falsos negativos (Recall=1.0 para la clase positiva), lo cual es crítico para detección de contaminaciones

3.4. Análisis de Errores

Del análisis de la matriz de confusión del Árbol de Decisión:

- **Falsos Positivos (1):** Una imagen sin arroz fue clasificada como con arroz
 - Posible causa: Ruido en la imagen o similitud con patrones de granos
 - Impacto: Bajo (solo 1 error)
- **Falsos Negativos (0): No se produjeron falsos negativos**
 - Todas las imágenes con arroz fueron correctamente identificadas
 - Esto es excelente para una aplicación de control de calidad

3.5. Limitaciones del Estudio

Debido al tamaño reducido del dataset (27 muestras únicas), los resultados deben interpretarse con cautela:

- El conjunto de prueba fue de solo 6 muestras, lo cual limita la significancia estadística
- Se recomienda aumentar el dataset para validar la generalización del modelo
- Los accuracy de 1.0 observados en versiones anteriores del código se debieron a data leakage (solapamiento train-test) y duplicados

4. Exportación del Modelo

4.1. Formato de Exportación

El modelo seleccionado (Árbol de Decisión) fue exportado junto con el transformador PCA utilizando la biblioteca `joblib` de Python, siguiendo las especificaciones del proyecto:

```
1 import joblib
2
3 # Empaquetar modelo y PCA
4 model_package = {
5     'classifier': best_model,
6     'pca': pca,
7     'model_name': 'Decision_Tree',
8     'metrics': metrics
9 }
10
11 # Exportar
12 filename = 'C10153_Luis_Aguirre.joblib'
13 joblib.dump(model_package, filename)
```

Listing 4: Código de exportación del modelo

4.2. Información del Archivo

- **Nombre del archivo:** C10153_Luis_Aguirre.joblib
- **Formato:** Joblib (serialización eficiente de objetos Python)
- **Contenido:**
 - Clasificador: Decision Tree Classifier
 - Transformador PCA (5 componentes)
 - Metadatos del modelo
- **Modelo:** Árbol de Decisión con profundidad máxima 3
- **Bibliotecas requeridas:** scikit-learn, joblib, numpy

4.3. Uso del Modelo Exportado

Para utilizar el modelo exportado en inferencia:

```
1 import joblib
2 import numpy as np
3
4 # Cargar el modelo (incluye PCA)
5 model_package = joblib.load('C10153_Luis_Aguirre.joblib')
6 classifier = model_package['classifier']
7 pca = model_package['pca']
8
9 # Preparar datos de entrada (imagen preprocesada)
10 # Debe ser un vector de 16384 elementos (128x128 binarizada)
11 imagen = np.loadtxt('imagen_vector.csv') # 16384 elementos
12
13 # Aplicar PCA
14 imagen_reducida = pca.transform([imagen])
15
16 # Hacer prediccion
17 prediccion = classifier.predict(imagen_reducida)
18
19 if prediccion[0] == 1:
20     print("ALERTA: Contaminacion detectada (arroz presente)")
21 else:
22     print("OK: No se detectaron contaminaciones")
```

Listing 5: Carga y uso del modelo exportado

4.4. Requisitos del Sistema

Para ejecutar el modelo exportado se requiere:

- Python 3.8 o superior

- `scikit-learn` $\geq$ 1.3.0
- `numpy` $\geq$ 1.24.0
- `joblib` $\geq$ 1.3.0

Instalación:

```
1 pip install scikit-learn numpy joblib
```

5. Conclusiones

5.1. Logros del Proyecto

1. Se entrenaron exitosamente cuatro modelos de clasificación clásica (Árbol de Decisión, Naive Bayes, KNN, SVM) con optimización de hiperparámetros
2. El modelo Árbol de Decisión alcanzó un accuracy de 83.33 % y F1-Score de 0.8571, demostrando ser efectivo para la detección de contaminaciones
3. La aplicación de PCA redujo la dimensionalidad de 16,384 a 5 componentes, explicando el 53.88 % de la varianza total y mejorando la eficiencia computacional
4. Se identificó y corrigió un problema de data leakage (solapamiento train-test) que inicialmente producía accuracy artificiales de 1.0
5. El modelo fue exportado correctamente en formato `.joblib` siguiendo la convención de nomenclatura `carne_nombre_apellido.joblib`

5.2. Recomendaciones para Trabajo Futuro

- **Aumentar el dataset:** Recolectar más imágenes únicas (mínimo 150 muestras totales) para mejorar la generalización
- **Data augmentation:** Aplicar rotaciones, traslaciones y cambios de brillo para aumentar artificialmente el dataset
- **Optimización adicional:** Explorar técnicas de balanceo de clases (SMOTE) dada la ligera desproporción (55.6 % / 44.4 %)
- **Ensemble methods:** Combinar múltiples árboles (Random Forest) para mejorar robustez
- **Deep Learning:** Evaluar redes neuronales convolucionales (CNNs) como trabajo futuro para comparar con aprendizaje clásico

5.3. Lecciones Aprendidas

- La importancia de validar que no haya solapamiento entre conjuntos de entrenamiento y prueba
- La necesidad de eliminar duplicados en datasets colaborativos
- El valor de la reducción de dimensionalidad (PCA) cuando se tienen pocas muestras y muchas features
- La interpretabilidad de los árboles de decisión como ventaja frente a modelos de caja negra

6. Referencias

1. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
4. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
5. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.
6. Documentación oficial de scikit-learn: <https://scikit-learn.org/>

A. Especificaciones del Hardware Utilizado

Componente	Especificación
Procesador	Intel Core i7 / AMD Ryzen
RAM	16 GB
Almacenamiento	SSD 512 GB
Sistema Operativo	Windows 11
Python	3.11

Cuadro 3: Especificaciones del equipo de desarrollo

B. Tiempos de Ejecución

Etapas	Tiempo (segundos)
Carga y limpieza de datos	0.2
Aplicación de PCA	0.3
Árbol de Decisión (GridSearch)	0.5
Naive Bayes	0.1
KNN (GridSearch)	0.4
SVM (GridSearch)	0.6
Exportación del modelo	0.1

Cuadro 4: Tiempo de ejecución por etapa

C. Metadatos del Modelo Exportado

```

1 =====
2 MODELO DE CLASIFICACION - PROYECTO 1 IE0435
3 =====
4
5 Estudiante: Luis Aguirre
6 Carne: C10153
7
8 MEJOR MODELO: Decision Tree
9 -----
10 Accuracy: 0.8333
11 Precision: 0.8571
12 Recall: 0.8571
13 F1-Score: 0.8571
14
15 Hiperparametros:
16   - criterion: gini
17   - max_depth: 3

```

```
18 - min_samples_split: 5
19
20 PCA:
21 - Componentes: 5
22 - Varianza explicada: 53.88%
```

Listing 6: Contenido del archivo C10153_{LuisAguirre}-metadata.txt